

Temeraire: Redis-Reproduktion mit öffentlichem Code

Von der hugepage-aware Idee bis zum funktionierenden Experiment

Aldo Costa

Heinrich-Heine-Universität Düsseldorf

Juli 2026

Hallo zusammen. Ich beschäftige mich heute mit Temeraire, einem Hugepage-bewussten Speicher-Allocator aus einem OSDI-Paper von 2021.

Mein ursprüngliches Ziel war relativ einfach formuliert: Ich wollte das öffentlich beschriebene Redis-Experiment reproduzieren. Im Laufe der Arbeit stellte sich allerdings heraus, dass das Ausführen von `redis-benchmark` der kleinste Teil des Problems war. Ein großer Teil der Arbeit bestand darin, überhaupt sicherzustellen, dass ich den richtigen Allocator, aktive Hugepages und die richtige Release-Konfiguration messe.

Was wird hier reproduziert?

Temeraire

Hugepage-bewusster Backend-Allocator für TCMalloc

Hunter et al., OSDI 2021

- Ziel im Paper: weniger TLB-Druck durch bessere Platzierung
- Ziel hier: der öffentliche Redis 6.0.9-Case-Study-Teil
- Kein Google-Fleet-Experiment, keine Produktions-Telemetrie

Arbeitsfrage

Kann die Redis-Methode mit öffentlichem Code und lokaler Hardware so nachgebaut werden, dass die kleine Effektgröße des Papers sichtbar wird?

Was wird hier reproduziert?

Temeraire ist ein alternatives Backend für TCMalloc, also für den Speicherverwalter, den ein Programm statt der üblichen Systembibliothek verwenden kann. Es soll Speicher so platzieren, dass Hugepages besser genutzt werden und dadurch weniger Druck auf den TLB entsteht. Der TLB ist ein kleiner Prozessor-Cache für bereits bekannte Adressübersetzungen.

Ich reproduziere hier ausdrücklich nur den öffentlichen Redis-Teil des Papers. Die Google-Fleet-Experimente, der Produktions-Rollout und die interne Telemetrie sind nicht öffentlich verfügbar.

Meine konkrete Frage war deshalb: Kann ich die Redis-Methode mit öffentlichem Code und lokaler Hardware so nachbauen, dass der sehr kleine Effekt des Papers sichtbar wird?

Hugepages als Ausgangspunkt

- Normale Seiten: 4 KiB
- Hugepages: 2 MiB, also 512 normale Seiten
- Ein Hugepage-TLB-Eintrag deckt viel mehr Speicher ab
- Der Gewinn ist weniger Adressübersetzung, nicht ein größerer Cache

Problem

Hugepages helfen nur, wenn der Speicher passend ausgerichtet und zusammenhängend bleibt.

Allocator-Rolle

`malloc` entscheidet indirekt, ob live Objekte eine Hugepage erhalten, fragmentieren oder für die Freigabe blockieren.

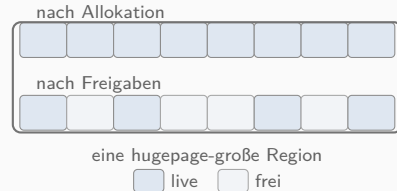
Hugepages als Ausgangspunkt

Normalerweise verwaltet Linux Speicher in Seiten von vier KiB. Eine Hugepage ist zwei MiB groß und deckt damit 512 normale Seiten ab.

Der Vorteil ist, dass ein einzelner TLB-Eintrag wesentlich mehr Speicher adressieren kann. Es geht also nicht um einen größeren Cache, sondern um weniger Arbeit bei der Übersetzung virtueller in physische Adressen.

Das funktioniert allerdings nur gut, wenn die Objekte passend im Speicher angeordnet werden. Genau an dieser Stelle wird der Allocator relevant.

Fragmentierung innerhalb einer Hugepage



- Freier Speicher reicht nicht aus, wenn er zwischen live Objekten liegt
- Teilfreigabe kann RAM sparen, aber Hugepage-Abdeckung zerstören
- Nichts freigeben kann TLB-Abdeckung halten, kostet aber Speicher

Temeraire macht diesen Trade-off explizit.

Es packt kleine Objekte bewusst auf schon teilgefüllte Hugepages.

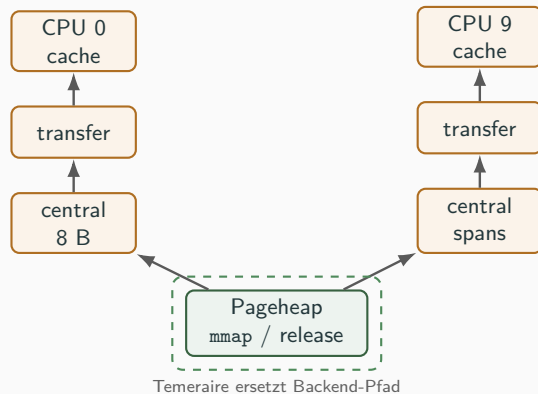
Fragmentierung innerhalb einer Hugepage

Hier sieht man eine vereinfachte Hugepage-Region. Oben ist die Region vollständig belegt. Unten wurden einige Objekte wieder freigegeben.

Obwohl jetzt freier Speicher vorhanden ist, kann die gesamte Hugepage nicht einfach als leer betrachtet werden. Es liegen weiterhin aktive Objekte dazwischen.

Damit entsteht ein Zielkonflikt: Gebe ich einzelne Seiten an das System zurück, kann ich die Hugepage-Abdeckung zerstören. Behalte ich alles, nutze ich mehr physischen Speicher. Temeraire versucht, neue kleine Objekte gezielt auf bereits teilweise belegte Hugepages zu packen.

Wo Temeraire in TCMalloc eingreift



- Obere Caches bleiben TCMalloc
- Temeraire ersetzt den Backend-Pageheap
- Reproduktion vergleicht deshalb Legacy Pageheap gegen den hugepage-aware Pfad

Wo Temeraire in TCMalloc eingreift

TCMalloc besteht aus mehreren Ebenen. Oben liegen CPU-lokale und zentrale Caches. Diese schnellen Pfade verändert Temeraire nicht.

Temeraire ersetzt den Backend-Pageheap. Das ist vereinfacht der Vorrat größerer Speicherseiten hinter den schnellen Caches: Er fordert neue Bereiche vom Betriebssystem an und gibt ungenutzten Speicher wieder zurück.

Der relevante Vergleich ist deshalb nicht glibc gegen TCMalloc. Ich muss denselben historischen TCMalloc-Code einmal mit dem alten Pageheap und einmal mit dem Temeraire-Backend bauen.

Vier Komponenten teilen die Anfragen nach Größe auf



Routing: $< 1 \text{ MiB} \rightarrow \text{HugeFiller}$; $1 \text{ MiB} - 1 \text{ GiB} \rightarrow \text{HugeCache} / \text{HugeRegion}$; $\geq 1 \text{ GiB} \rightarrow \text{HugeCache}$. Redis trifft vor allem den kleinen Pfad.

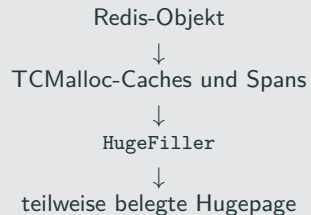
Vier Komponenten teilen die Anfragen nach Größe auf

Das Temeraire-Backend besteht hauptsächlich aus diesen vier Komponenten.

Der HugeAllocator holt ausgerichtete Hugepage-Bereiche vom System. Der HugeCache bewahrt leere, bereits physisch hinterlegte Hugepages zur Wiederverwendung auf. Der HugeFiller packt kleine Allokationen auf teilweise belegte Hugepages. HugeRegion behandelt mittlere Allokationen, bei denen sonst zu viel ungenutzter Rest entstehen könnte.

Für den Redis-Workload ist vor allem HugeFiller interessant, weil Redis hier viele kleine Listen- und String-Objekte erzeugt.

🕸 Weg einer kleinen Allokation



Wichtige Grenze

HugeFiller sieht keine Redis-Listen direkt. Er platziert die kleinen Spans hinter den schnellen TCMalloc-Caches.

⚡ Seine eigentliche Aufgabe

- freie Bereiche in bereits belegten Hugepages nutzen
- kleine Spans dichter zusammenhalten
- möglichst wenige Hugepages nur teilweise belegen

Warum das bei Redis helfen kann

Dichtere Belegung kann mehr aktiven Speicher mit Hugepage-TLB-Einträgen abdecken und leere Hugepages leichter freigeben.

Von den vier Komponenten ist für diesen Redis-Workload vor allem der HugeFiller relevant. Dabei darf man den Weg aber nicht zu stark verkürzen: Der HugeFiller verwaltet nicht direkt einzelne Redis-Strings oder Listenknoten.

Redis fordert kleine Speicherblöcke an. Zuerst bedienen die schnellen TCMalloc-Schichten diese Anfragen aus CPU-lokalen oder zentralen Caches. Wenn diese Schichten neue kleine Spans aus dem Backend brauchen, kommt der HugeFiller ins Spiel. Ein Span ist hier ein zusammenhängender Bereich aus Speicherseiten, aus dem TCMalloc gleich große Objekte bedient.

Der HugeFiller versucht, solche kleinen Spans in freie Bereiche bereits teilweise belegter Hugepages zu setzen. Vereinfacht gesagt verteilt er sie nicht unnötig auf viele halbvolle Hugepages. Dadurch kann ein größerer Anteil des aktiven Speichers durch Hugepage-TLB-Einträge abgedeckt werden. Gleichzeitig steigt die Chance, dass andere Hugepages ganz leer bleiben und später zurückgegeben werden können.

Genau daraus entsteht die Redis-Hypothese der nächsten Folie: Viele kleine Allokationen erzeugen laufend Bedarf an diesen kleinen Spans. Wenn ihre Platzierung dichter ist, sollte sich das als kleiner Durchsatzvorteil zeigen. Ob dieser Effekt auf meinem Host tatsächlich sichtbar ist, ist dann die experimentelle Frage.

Workload aus dem Paper

- Redis 6.0.9
- LPUSH von fünf Werten
- LRange 0 4
- 2000 Trials, je 1 Mio. Requests

Erwarteter Effekt

Redis erzeugt viele kleine Listen- und String-Allokationen. Wenn HugeFiller sie besser platziert, sinkt der TLB-Druck.

$$\text{kombiniert} = \frac{2}{1/r_{\text{LPUSH}} + 1/r_{\text{LRange}}}$$

Konsequenz für die Messung

Der erwartete Unterschied liegt unter einem Prozent. Reihenfolge, THP-Zustand und Laufzeit-Validierung dürfen nicht implizit bleiben.

Das Paper verwendet Redis 6.0.9. Der Workload schreibt mit LPUSH fünf Werte in eine Liste und liest sie anschließend mit LRange 0 4.

Pro Allocator-Modus werden 2.000 Trials ausgeführt, mit jeweils einer Million Requests. Die Schreib- und Leserate kombiniere ich über das harmonische Mittel.

Die Hypothese ist: Wenn Temeraire diese kleinen Objekte günstiger auf Hugepages verteilt, sinkt der TLB-Druck und die Redis-Rate steigt leicht.

Das entscheidende Wort ist leicht. Der erwartete Unterschied liegt unter einem Prozent. Schon relativ kleine Mess- oder Konfigurationsfehler können also größer sein als der gesuchte Effekt.

Der Vergleich hat zwei getrennte Betriebsarten

Release-off

- keine periodische Pageheap-Freigabe
- freier Speicher bleibt länger beim Allocator
- je ein Legacy- und Temeraire-Run

Release-on

- Background-Thread gibt Speicher periodisch zurück
- dafür ist eine positive Rate nötig
- je ein Legacy- und Temeraire-Run

Zwei Dimensionen, nicht zwei Allocatoren

Der Backend-Vergleich bleibt immer **Legacy gegen Temeraire**. Er wird einmal ohne und einmal mit periodischer Freigabe gemessen. Das Paper nennt für Release-on keine konkrete Rate.

Der Vergleich hat zwei getrennte Betriebsarten

Bevor ich den technischen Aufbau erkläre, ist eine zweite Unterscheidung wichtig. Das Paper betrachtet den Redis-Vergleich in zwei getrennten Betriebsarten: Release-off und Release-on.

Release-off bedeutet, dass TCMalloc nicht regelmäßig ungenutzte Pageheap-Seiten an das Betriebssystem zurückgibt. Bei Release-on übernimmt das ein Background-Thread. Dafür reicht es nicht, den Thread nur zu starten; es muss auch eine positive Freigaberate gesetzt sein.

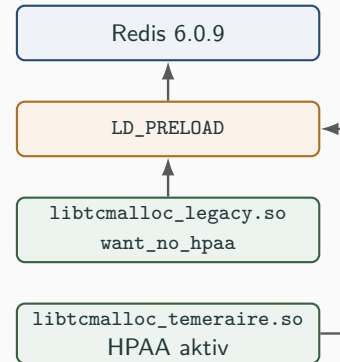
Diese Release-Einstellung ist unabhängig von der Allocator-Wahl. In jeder Betriebsart brauche ich daher ein zusammengehöriges Paar aus Legacy und Temeraire. Die späteren Ergebnisfolien behandeln zuerst Release-off und danach die korrigierten Release-on-Messungen.

Zwei Libraries machen den Vergleich möglich

Rolle der Wrapper

Redis wird nicht zweimal umgebaut. Stattdessen werden zwei kleine TCMalloc-Shared-Libraries gebaut und beim Start in den Prozess geladen.

- gleicher Redis-Build
- gleiche Wrapper-Quelle
- unterschiedliche TCMalloc-Backend-Konfiguration



Abgrenzung

Der Wrapper ist kein neuer Allocator. Er macht den historischen TCMalloc-Code preloadbar, damit der Legacy-vs.-Temeraire-Pfad messbar wird.

Zwei Libraries machen den Vergleich möglich

Redis wird nicht zweimal unterschiedlich kompiliert. Ich baue Redis einmal und erzeuge zwei TCMalloc-Shared-Libraries.

Beim Start von Redis wird über LD_PRELOAD entweder die Legacy- oder die Temeraire-Library geladen. LD_PRELOAD ist eine Linux-Funktion, mit der sich eine zusätzliche Programmbibliothek noch vor den normalen Bibliotheken laden lässt. Eine Shared Library ist dabei wiederverwendbarer, bereits kompilierter Code, den Redis beim Start einbindet. Dadurch bleiben Redis-Binary, Benchmark und Wrapper-Quelle gleich. Nur die Backend-Konfiguration ändert sich.

Der Wrapper ist dabei kein neuer Allocator. Er ist nur eine dünne Anpassungsschicht, die den historischen TCMalloc-Code als preloadbare Library verfügbar macht.

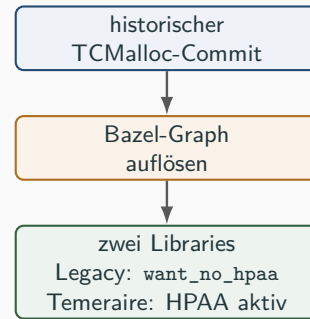
Bazel baut die zwei TCMalloc-Varianten

Rolle von Bazel

Bazel ist das Build-System von TCMalloc. Es kennt dessen Quellcode, Abhängigkeiten und Link-Regeln.

- TCMalloc ist selbst ein Bazel-Projekt
- Abseil und interne Targets kommen aus dem nativen Build-Graph
- `want_no_hpaa` ist ein TCMalloc-Target, kein Runtime-Flag

Bazel brachte TCMallocs Build-Graphen mit: Abseil, interne Targets und Link-Semantik mussten nicht in CMake nachgebaut werden.



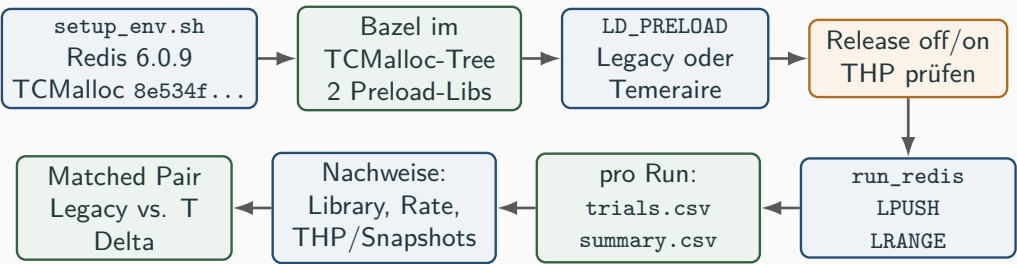
Bazel baut die zwei TCMalloc-Varianten

Bazel ist das Build-System von TCMalloc, also das Werkzeug, das aus dem Quellcode und seinen Abhängigkeiten die fertigen Bibliotheken erzeugt. Dabei kennt es nicht nur einzelne Dateien, sondern den gesamten Abhängigkeitsgraphen des Projekts.

Der Wrapper wird deshalb direkt im historischen TCMalloc-Quellbaum gebaut. So kann er die vorhandenen internen Bausteine, Abseil-Abhängigkeiten und Link-Regeln verwenden. Abseil ist dabei eine Sammlung von C++-Grundbibliotheken, auf die TCMalloc aufbaut.

Für die Legacy-Library wird zusätzlich das Build-Ziel `want_no_hpaa` eingebunden. HPAA steht hier für den Hupage-aware Allocator, also Temeraire. Dieses Ziel deaktiviert den neuen Backend-Pfad; die andere Library behält ihn aktiv. Damit ist jetzt der komplette Sollaufbau erklärt. Als Nächstes zeige ich, wie ein Messpaar dadurch laufen sollte.

So sollte ein gültiges Messpaar ablaufen



 Umgebung	 Pro Run	 Pro Messpaar
• Docker: Debian 12 (bookworm) • WSL2-Kernel 6.6.114.1, x86_64 • Host-CPU: Intel i7-10700K	• 2000 Trials; je 1 Mio. Requests • genau eine Library und ein Release-Regime • Library, Rate und THP bestätigen	• gleiche Parameter und Release-Einstellung • Legacy und Temeraire getrennt • danach Delta berechnen

So sollte ein gültiges Messpaar ablaufen

Das ist die Sollvorstellung, auf die ich mich bei den folgenden Problemen immer wieder beziehe.

Die technische Umgebung ist dabei klar getrennt: Docker stellt mit Debian 12, genauer dem Image `debian:bookworm-slim`, den User Space und die Werkzeuge bereit. Der Container besitzt aber keinen eigenen Linux-Kernel. Er verwendet den Kernel `6.6.114.1-microsoft-standard-WSL2` des WSL2-Unterbaus auf dem Windows-Rechner. Die Architektur ist `x86_64`; als CPU ist ein Intel Core i7-10700K sichtbar. Das Projektverzeichnis ist als `/workspace` eingebunden, deshalb bleiben Skripte, Logs und Resultate auf dem Host erhalten.

Diese Grenze ist für die Messung wichtig: Build-Werkzeuge und Bibliotheken kommen aus dem Debian-Container. Kernelverhalten, THP, Scheduling und die sichtbare Hardware kommen dagegen von WSL2 und dem Host. Der Container macht den Softwareaufbau reproduzierbarer, isoliert die Messung aber nicht von diesen Host-Einflüssen.

Zuerst baut `setup_env.sh` Redis und die beiden TCMalloc-Libraries. Für einen einzelnen Run wähle ich genau eine Library, also Legacy oder Temeraire, und genau ein Release-Regime. Bei Release-on gehört dazu eine positive Rate. Außerdem muss THP aktiv sein. THP steht für Transparent Huge Pages: Linux versucht dabei automatisch, normalen Programmspeicher als Hugepages abzubilden.

Danach führt der Runner die 2.000 Trials mit LPUSH und LRANGE aus. Gleichzeitig speichert er die Rohdaten und die Nachweise dafür, welche Library geladen war, welche Release-Rate galt und ob Redis tatsächlich Hugepages nutzte.

Erst am Ende werden zwei Runs mit denselben Parametern und demselben Release-Regime als Legacy/Temeraire-Paar verglichen. Die nächsten Folien zeigen nun, an welchen Stellen dieser eigentlich einfache Ablauf zunächst nicht zuverlässig erfüllt war.

Erster Hiccup: Der Build war nicht übertragbar

Was nicht sauber funktionierte

- Moderner TCMalloc-Stand passte nicht zum historischen Wrapper
- Externer Wrapper erhielt nicht alle internen Abhängigkeiten
- Legacy-Target war außerhalb des Build-Graphen nicht sichtbar

Was am Ende funktionierte

- Wrapper im historischen TCMalloc-Quellbaum
- passende Bazel-Version festlegen
- denselben Compiler für alle Build-Schritte verwenden

Erster Hiccup: Der Build war nicht übertragbar

Mit diesem Sollablauf im Kopf beginnt jetzt der eigentliche Reproduktionsweg. Der erste technische Hiccup lag noch vor jeder Messung und lässt sich einfacher erklären, als die Folie zunächst aussieht.

Mein erster Ansatz behandelte den Wrapper als eigenes Bazel-Projekt neben TCMalloc. Das klingt zunächst vernünftig, aber TCMalloc war bereits ein vollständiges Bazel-Projekt. Das zweite Projekt erbte TCMallocs interne Abhängigkeiten, Link-Regeln und Zugriffsrechte nicht automatisch. Besonders das Legacy-Ziel `want_no_hpaa` war von außen nicht zugänglich.

Die Lösung war deshalb nicht, einen komplizierteren zweiten Bazel-Aufbau zu bauen. Stattdessen kopiert das Setup die Wrapper-Datei beim Build in den historischen TCMalloc-Quellbaum und trägt sie dort als zwei normale TCMalloc-Build-Ziele ein. Der Wrapper wird damit Teil von TCMallocs bereits vorhandenem Bazel-Projekt und kann dessen Abhängigkeiten und das interne Legacy-Ziel verwenden.

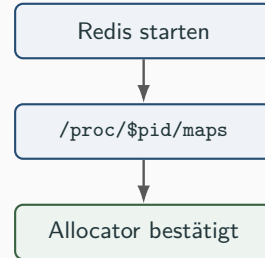
Wichtig ist: Die eigentliche Wrapper-Quelldatei bleibt in unserem Repository. Sie wird nur für den Build nach `tcmalloc/testing` kopiert. Danach mussten noch die passende Bazel-Version und derselbe Compiler für alle Build-Schritte festgelegt werden. Das Grundproblem war aber die unnötige Trennung in zwei Bazel-Projekte.

Ein erfolgreicher Start beweist noch nicht den Allocator

- Redis kann weiter `mem_allocator: libc` melden
- Diese Meldung kommt aus der Redis-Build-Konfiguration
- Für `LD_PRELOAD` zählt, was der Prozess wirklich mappt

Prüfung

`/proc/$pid/maps` muss die passende Library zeigen:
`libtcmalloc_legacy.so` oder
`libtcmalloc_temeraire.so`.



Ein erfolgreicher Start beweist noch nicht den Allocator

Nachdem der Build stand, kam der zweite Hiccup an der nächsten Stelle des Ablaufs: Ein gestarteter Redis-Prozess beweist noch nicht, welche Library er wirklich verwendet.

Redis kann weiterhin `mem_allocator: libc` ausgeben, obwohl eine TCMalloc-Library über `LD_PRELOAD` geladen wurde. Diese Anzeige beschreibt hauptsächlich, wie Redis gebaut wurde.

Deshalb prüft das Artefakt `/proc/<pid>/maps`. Dort muss die tatsächlich gemappte Library erscheinen. Erst wenn dort entweder `libtcmalloc_legacy.so` oder `libtcmalloc_temeraire.so` steht, ist der Allocator für diesen Prozess bestätigt.

Ein erfolgreicher Benchmark allein wäre hier also kein ausreichender Nachweis.

Beobachtung in frühen Runs

```
AnonHugePages: 0kB
```

Redis lief, die Allocator-Varianten liefen, aber der Hugepage-Mechanismus war nicht belegt.

Fix im Harness

- `thp-before.txt` und `thp-after.txt`
- `memory-sample-0250..2000.txt`
- `smaps_rollup` mit `AnonHugePages`
- nur THP-aktive Runs für das Paper-Signal verwenden

Kriterium für gültige Runs

Spätere Runs liefen mit `[always] madvise never`; Redis-Snapshots zeigten 14–18 MiB `AnonHugePages` im Prozess. Erst dann war der Hugepage-Pfad wirklich Teil der Messung.

Die Library war nun bestätigt. Der nächste Prüfpunkt aus dem Sollablauf war der Mechanismus selbst: Nutzt Redis während des Runs überhaupt Hugepages?

Die Allocator-Libraries wurden korrekt geladen und Redis lief, aber die Speicher-Snapshots zeigten `AnonHugePages: 0kB`. Der Mechanismus, den Temeraire verbessern soll, war damit gar nicht aktiv.

Daraufhin habe ich den Runner um THP-Snapshots und regelmäßige `smaps_rollup`-Messungen erweitert. `smaps_rollup` ist eine zusammengefasste Speicherübersicht des laufenden Prozesses. Nach der Korrektur lief die THP-Policy auf `always`, und Redis zeigte ungefähr 14 bis 18 MiB `AnonHugePages`.

Erst diese Runs wurden für den eigentlichen Paper-Vergleich verwendet.

Release-on war faktisch nicht eingeschaltet

▶ Was lief

- Background-Thread gestartet
- Per-CPU-Caches wurden gepflegt

⚠ Was fehlte

Release-Rate nicht gesetzt: TCMalloc blieb bei 0 B/s. Damit gab es keine periodische Pageheap-Freigabe.

Konsequenz

Die alten Werte waren kein Release-on-Ergebnis. Der korrigierte Lauf setzt eine positive Rate explizit; das Paper nennt seine Rate nicht.

Release-on war faktisch nicht eingeschaltet

Nach Library und Hugepages blieb noch der zweite Betriebsmodus aus der Setup-Folie zu prüfen: Release-on. Genau dort lag der nächste Hiccup.

Der TCMalloc-Background-Thread wurde zwar gestartet, aber die eigentliche Release-Rate wurde nicht gesetzt. Im historischen Commit bedeutet das eine Standardrate von null Byte pro Sekunde.

Der Thread verwaltete weiterhin bestimmte Caches, gab aber keine Pageheap-Bytes periodisch an das System zurück. Die bisherigen Runs mit dem Namen `release-on` waren deshalb nicht wirklich Release-on-Runs.

Der korrigierte Wrapper verlangt nun eine positive Rate und bestätigt sie explizit im Redis-Log.

Paar	Reihenfolge	Delta
Paar 1	L first	+1.88%
Paar 2	L first	+0.43%
Paar 3	T first	+0.48%
Paar 4	L first	-0.76%
Paar 5	T first	+3.46%

Befund

Vier von fünf THP-aktiven vollständigen Runs favorisieren Temeraire. Der Median liegt bei +0.48%.

Paper-Vergleich

Redis[†] im Paper: +0.75%
Periodische Freigabe deaktiviert.

Tabelle lesen: Jede Zeile ist ein Legacy/Temeraire-Paar. Ein positives Delta bedeutet: Temeraire war schneller.

Damit sind die Voraussetzungen und ihre drei wichtigsten Fehlerstellen geklärt: Build, geladene Library und Laufzeitkonfiguration. Ich trenne die Ergebnisse nun wie angekündigt nach Release-Regime und beginne mit Release-off, weil dort keine positive Release-Rate konfiguriert werden muss.

Zuerst zur Tabelle. Jede Zeile steht für ein zusammengehöriges Legacy/Temeraire-Paar. Die Spalte Reihenfolge zeigt, welcher Allocator zuerst lief: L first bedeutet Legacy zuerst, T first Temeraire zuerst. Das Delta ist immer Temeraire relativ zu Legacy. Ein positiver Wert bedeutet also, dass Temeraire in diesem Paar schneller war; ein negativer Wert bedeutet, dass Legacy schneller war.

Als Beispiel kann man Paar 3 lesen: Temeraire lief zuerst und war gegenüber dem dazugehörigen Legacy-Run um 0,48 Prozent schneller. Insgesamt favorisieren vier von fünf vollständigen, THP-aktiven Paare Temeraire. Der Median der fünf Deltas liegt ebenfalls bei plus 0,48 Prozent.

Rechts steht dann der direkte Vergleich zum Paper: Dort beträgt der Redis-Effekt bei deaktivierter periodischer Freigabe plus 0,75 Prozent. Die lokale Größenordnung passt ungefähr, aber die einzelnen Paare reichen von minus 0,76 bis plus 3,46 Prozent. Deshalb würde ich nicht sagen, dass der exakte Paper-Wert reproduziert wurde.

Release-on zeigt keinen stabilen Paper-Effekt

Rate	Paar 1	Paar 2	Paar 3	Paar 4	Median
16 MiB/s	+1.91%	-0.16%	-0.27%	-1.37%	-0.21%
64 MiB/s	-6.88%	+2.10%	+11.49%	+12.21%	+6.80%
256 MiB/s	+15.73%	-0.09%	+0.50%	-0.77%	+0.20%

Tabelle lesen: Jede Zelle ist das Delta eines Legacy/Temeraire-Paars. Positiv bedeutet: Temeraire war schneller.

Lokaler Befund

Mediane: -0.21%, +6.80%, +0.20%. Die drei Raten ergeben keinen monotonen oder stabilen Trend.

Paper-Vergleich

Redis[†] im Paper: +0.44%. Die lokale Matrix reproduziert diesen kleinen Effekt nicht stabil.

Release-on zeigt keinen stabilen Paper-Effekt

Die alten, falsch bezeichneten Release-on-Runs bleiben zwar als Fehlerprotokoll im Repository, ihre Deltas werden aber nicht als Ergebnis verwendet. Diese Tabelle enthält nur die korrigierten Messungen mit positiver, im Server-Log bestätigter Release-Rate.

Auch hier beginne ich mit der Leserichtung der Tabelle. Jede Zeile ist eine getestete Release-Rate: 16, 64 oder 256 MiB pro Sekunde. Die Spalten Paar 1 bis Paar 4 sind vier getrennte Legacy/Temeraire-Paare mit wechselnder Reihenfolge. Jede Zelle verwendet dieselbe Konvention wie auf der vorherigen Folie: positiv bedeutet Temeraire schneller, negativ bedeutet Legacy schneller. Die letzte Spalte ist der Median der vier Paare dieser Rate.

Die erste Zeile kann man so lesen: Bei 16 MiB pro Sekunde ist Temeraire im ersten Paar 1,91 Prozent schneller, in den drei folgenden Paaren aber leicht langsamer. Deshalb liegt der Median dieser Zeile bei minus 0,21 Prozent. Insgesamt umfasst die Matrix 24 Allocator-Runs mit jeweils 2.000 Trials.

Danach vergleiche ich mit dem Paper. Dort liegt der Release-on-Effekt für Redis bei plus 0,44 Prozent. Lokal liegen die Mediane bei minus 0,21, plus 6,80 und plus 0,20 Prozent. Besonders die 64-MiB-Reihe streut von minus 6,88 bis plus 12,21 Prozent.

Es gibt damit weder einen monotonen Zusammenhang mit der Release-Rate noch eine stabile Wiederholung des kleinen Paper-Effekts. Die nächste Folie erklärt, warum die großen Ausschläge eher zur Host-Drift als zur Release-Rate passen.

Host-Drift ist größer als der erwartete Effekt

Auswertbarer Befund

Release-off liegt mit +0.48% Median im kleinen Effektbereich des Paper-Werts von +0.75%. Die fünf Run-Paare streuen jedoch deutlich.

Korrigierter Befund

Release-on ist technisch korrekt gemessen, zeigt aber keinen stabilen Rate-Trend. Host-Drift ist größer als der Paper-Effekt.

Vertretbare Aussage

Das Artefakt rekonstruiert den Redis-Vergleich mit öffentlichem Code. Der korrigierte Lauf zeigt vor allem die Grenze konsekutiver Ein-Stunden-Runs.

Host-Drift ist größer als der erwartete Effekt

Bei der Betrachtung der absoluten Durchsatzwerte wurde deutlich, dass die Host-Leistung während der langen Runs stark schwankte.

Die Gesamtleistung fiel zeitweise um mehr als 20 Prozent und erholte sich später wieder. Das ist wesentlich größer als der Effekt von ungefähr einem halben Prozent, den wir eigentlich suchen.

Dadurch können zwei direkt nacheinander ausgeführte Runs unterschiedlich aussehen, obwohl sich hauptsächlich der Zustand des Hosts verändert hat. Der Host ist hier mein Windows-Rechner; Redis selbst läuft in Docker innerhalb von WSL2, also einer Linux-Umgebung unter Windows. Mögliche Ursachen sind die Verteilung der Rechenzeit durch WSL2 und Docker, CPU-Takt und Temperatur, Windows-Hintergrundlast oder dynamische Speicherverwaltung.

Der korrigierte Release-on-Lauf zeigt daher vor allem die Grenze dieses Messaufbaus.

Was diese Reproduktion nicht zeigen kann

Nicht reproduziert

- Google-Fleet-Experiment
- Produktions-Rollout
- interne Allocator-Binaries
- exakte Paper-Hardware

Lokale Abweichungen

- Docker/WSL2 und geteilter Kernel
- Consumer i7 statt Skylake Xeon Server
- THP-Policy aus der Host/VM-Umgebung
- Public TCMalloc als historische Annäherung

Nutzen der Grenzen

Sie trennen den methodisch reproduzierten Redis-Pfad von den Teilen des Papers, die ohne Google-Infrastruktur nicht nachstellbar sind.

Was diese Reproduktion nicht zeigen kann

Die Reproduktion deckt nicht das gesamte Paper ab.

Nicht reproduziert wurden die Google-Fleet-Experimente, der Produktions-Rollout, interne Allocator-Binaries und die exakte Server-Hardware.

Zusätzlich läuft mein Aufbau unter Docker und WSL2 auf einer Consumer-CPU. THP und Kernel-Verhalten kommen aus der Host- beziehungsweise VM-Umgebung. Der öffentliche historische TCMalloc-Code ist außerdem nur eine Annäherung an Googles internes Artefakt.

Diese Grenzen machen den Aufbau nicht wertlos, aber sie begrenzen die Aussage: Ich kann den öffentlichen Redis-Pfad rekonstruieren, nicht die gesamte Produktionsbewertung.

Technische Lektion

Der schwere Teil war nicht `redis-benchmark`, sondern der Weg zu einem echten Legacy-vs.-Temeraire-Vergleich mit aktiven Hugepages.

- Vergleich aus public TCMalloc rekonstruiert
- Preload zur Laufzeit validiert
- THP-Fehlerbild gefunden und instrumentiert
- Release-on korrigiert und mit drei Raten neu gemessen

Endstand

Release-off zeigt ein kleines positives Mediansignal bei hoher Streuung. Release-on ist aktiv, aber Host-Drift verhindert ein stabiles Ergebnis.

Mein Fazit ist, dass der schwierige Teil nicht `redis-benchmark` war. Der schwierige Teil war, einen echten Legacy-gegen-Temeraire-Vergleich herzustellen und danach zu beweisen, dass der gewünschte Mechanismus wirklich aktiv ist.

Dafür mussten der historische TCMalloc-Pfad rekonstruiert, die geladene Library validiert, THP instrumentiert und die Release-on-Konfiguration korrigiert werden.

Release-off zeigt ein kleines positives Mediansignal in der Größenordnung des Papers, allerdings bei deutlicher Streuung. Release-on ist jetzt technisch korrekt aktiv, liefert auf diesem Host aber kein stabiles Ergebnis, weil die Host-Drift wesentlich größer als der erwartete Effekt ist.

Damit ist das Ergebnis weniger eine Bestätigung einer exakten Zahl und eher eine nachvollziehbare Reproduktion des experimentellen Weges – einschließlich der Stellen, an denen ein scheinbar funktionierender Lauf noch nicht wirklich das misst, was sein Name behauptet.